

Introducción a la Ciencia de Datos

Guía de trabajos prácticos N°2

Tabla de contenidos

Introducción	1
Análisis sobre el <i>dataset titanic</i>	2
Máximos / Mínimos / Valores únicos	3
Ordenar	4
Filtrar	5
Gráficos de Barras	6
Agrupar	11
Para trabajar en casa	14
Bibliografía obligatoria	14
Bibliografía para profundizar	15

Introducción

Les proponemos iniciar el análisis del conjunto de datos Titanic, disponible en el Campus de la materia, a partir de una serie de consignas diseñadas para trabajar en **RStudio**. Si lo necesitan, pueden repasar la Guía 1 para recordar el manejo básico de esta herramienta.

Análisis sobre el *dataset* titanic

En el campus de la materia encontrarán el *dataset* Titanic, que reúne información sobre todos los pasajeros que viajaban en el RMS Titanic en abril de 1912, cuando ocurrió su hundimiento. Descárguenlo para trabajar con él. Este conjunto de datos incluye, para cada pasajero, los siguientes atributos:

`sex` = Sexo del pasajero

`age` = Edad en años

`siblings.of.the.passenger` = Cantidad de hermanos/cónyuges a bordo

`parents/children.aboard.the.Titanic` = Cantidad de padres/hijos a bordo

`fare` = Tarifa abonada

`Port.of.Embarkation` = Puerto donde embarcó: C - Cherbourg, Q - Queenstown, S - Southampton

`class` = Clase del ticket

`who` = Man/Woman/Child

`alone` = TRUE: viajaba solo, FALSE: no viajaba solo

`survived` = TRUE: Sobrevivió / FALSE: No sobrevivió

Como ya vimos en la guía anterior vamos a usar “tidyverse” para visualizar tablas ordenadas y realizar gráficos en **RStudio**. Repasemos un poco como cargamos y empezamos a analizar un *dataset*.

1. Creen un nuevo script en **RStudio** y guárdenlo en el mismo lugar donde bajaron el *dataset*.
2. Carguen la biblioteca *tidyverse* con `library(tidyverse)` y lean el archivo con `read_csv` para cargarlo a un *data frame* (recordando que hay que poner la ruta completa al archivo o ir a *Session -> Set working directory -> To Source File Location*):

```
titanic_df <- read_csv("titanic.csv", locale = locale(decimal_mark = ","))
```

¡Atención!

El argumento `locale = locale(decimal_mark = ",")` le indica a **R** que, dentro del archivo .CSV, los números utilizan **coma** como separador decimal (por ejemplo, 3,14 en lugar de 3.14). Esto es importante porque, si no se especifica correctamente,

R podría interpretar esos valores como texto o leerlos de forma incorrecta (si quieren pueden probar no usarlo y vean que pasa).

Este ajuste se debe a la forma en que se guardaron los datos en **este archivo** en particular; no siempre es necesario aplicarlo.

3. Pueden visualizar los datos como una tabla usando el comando `view(...)`.
¿Cuáles son las unidades de estudio?
4. Aplicando los comandos `spec(...)` y `glimpse(...)` se puede obtener información sobre la tabla de datos.
 - a. ¿Qué diferencias hay entre los *resultados* del comando `spec(...)` y `glimpse(...)`? ¿Qué información comparten?
 - b. ¿Cuántas observaciones hay?
5. Utilizando la información que se obtiene aplicando los comandos anteriores, respondan:
 - a. ¿Cuáles son las variables del *dataset*? ¿De qué tipos son?
 - b. Muchas veces los tipos de las variables se cargan con valores por defecto como 'chr' (char). Para las variables anteriores determinar si el tipo cargado es correcto o si consideran que debería ser otro.
 - c. ¿Qué variables consideran que podrían ser categóricas?

Máximos / Mínimos / Valores únicos

Vamos a empezar a conocer el *dataset* y los posibles valores de las variables filtrando y ordenando las unidades. Recordemos que para obtener todos los valores de un atributo (o columna de tabla) usamos el símbolo "\$". Así, `titanic_df$columna_1` nos devuelve una lista con los valores de la columna_1 del *data frame*. Esto se puede usar en conjunción con otras funciones como `min(...)`, `max(...)` y `unique(...)` que extraen los valores **mínimo**, **máximo** o **elimina repetidos** de una lista de valores.

6. Seleccione todos los valores de la columna *age* (edad) ejecutando el comando `titanic_df$age`, van a ver que se los muestra todos en la consola, si hacen `min(titanic_df$age)` están ejecutando la función mínimo sobre todos esos números. Háganlo y traten de explicar el resultado.

NA (*Not Available*) significa que hay un dato faltante entre los valores de la columna. Si se fijan en una de las salidas anteriores pueden ver que no es solo uno, son muchos. Esto trae problemas cuando se quiere usar el mínimo sencillamente porque el elemento **NA** no es un número y no se puede calcular el mínimo si no sabemos qué valor tiene ese

elemento, lo mismo pasa con muchas otras funciones, lo que podemos hacer (por ahora) es indicarle que ignore esos valores agregando el argumento `na.rm=TRUE`. Por ejemplo:

```
min(titanic_df$age, na.rm=TRUE)
```

Tanto `min` como `max` son funciones del paquete `base` de R, que solo funcionan sobre una columna. El paquete `dplyr`, parte de `tidyverse`, tiene una serie de funciones, la familia `slice`, que permite obtener filas completas de un *dataset*. Por ejemplo, `slice_min` permite obtener la fila (o filas) en las que alguna variable toma su valor mínimo. `slice_max` funciona de manera similar.

7. Prueben usar `slice_min` con el *dataset*, fijando el argumento `order_by` a `age`: `slice_min(titanic_df, order_by=age)`, o simplemente `slice_min(titanic_df, age)`. Comparen el resultado con el que se obtuvo en el punto anterior. ¿Pueden decir ahora si la persona sobrevivió? **Ayuda:** pueden usar `relocate` para poner delante la columna `survived`: `relocate(slice_min(titanic_df, order_by=age), survived)`.
8. Hagan lo mismo con `slice_max`.
9. En alguno de los casos anteriores, agreguen el argumento `n` a la función `slice_max` o `slice_min`, y con algún valor mayor que 1. P.ej.: `slice_max(titanic_df, age, n=5)`. ¿Pueden describir qué hace ese argumento?
10. Con estas herramientas, respondan:
 - a. ¿Cuál fue el boleto más barato (con *fare* menor)? ¿Y el más caro?
 - b. ¿Cuáles eran los puertos desde donde embarcaron (*Port.of.Embarkation*)? (TIP: si presionan “TAB” **RStudio** los ayuda a autocompletar los nombres de *data frames* y columnas).
 - c. ¿Cuántas categorías de clase (*class*) había?

Ordenar

El comando `arrange` permite ordenar las filas de un *data frame* en función de una o varias columnas/variables. Por ejemplo, para ordenar las filas de forma ascendente por la variable *fare*, `arrange(titanic_df, fare)` y para hacerlo de forma descendente `arrange(titanic_df, desc(fare))`. Utilizando este comando responder:

11. ¿Desde qué puerto embarcaron las dos personas más jóvenes que estaban en el barco?
12. ¿De qué clase eran las dos personas más longevas?

Estas respuestas también pueden obtenerse con los comandos `slice` que describimos más arriba. Con el comando `arrange` también se puede ordenar por más de una variable. Para ello hay que pasarle las variables por las que queremos que ordene en el orden que queremos que lo ordene: `arrange(data, Variable1, Variable2)`. Prueben aplicando este comando al *dataset* de titanic ordenando primero por `fare`, después por `age` y viceversa. Vean la tabla que resulta en cada caso.

Filtrar

Otra función útil es *filter*, que sirve para quedarnos solamente con las unidades que cumplan con ciertas condiciones. Se ejecuta de la siguiente forma: `filter(data frame, condición)`, se le pasa el *data frame* y la condición que deben cumplir las unidades (atributo y valores permitidos). Por ejemplo, para quedarnos solamente con aquellos pasajeros de primera clase podemos ejecutar `filter(titanic_df, class=='First')`. Esto devuelve un *data frame* que sólo contiene aquellos pasajeros de primera clase.

Un detalle importante es que a diferencia de las funciones que vimos más arriba, **filter ignora los NA**.

Para no tener que calcular todo el tiempo el mismo filtro o la misma selección de columna se pueden guardar resultados intermedios en una variable y después usarlo. Esto se hace con la asignación `<-` (o con el atajo `Alt+-` (`alt` + `menos`)) por ejemplo, si queremos guardar el resultado del filtro de los pasajeros de primera clase para usarlo después o la lista de todas las edades:

```
titanic_first <- filter(titanic_df, class=='First')
titanic_age <- titanic_df$age
```

Esto guarda en `titanic_first` el *data frame* filtrado y en `titanic_age` las edades, después simplemente los usamos. Por ejemplo, para calcular la edad máxima de los pasajeros de primera clase y la mínima de todos:

```
max(titanic_first$age, na.rm=TRUE)
min(titanic_age, na.rm=TRUE)
```

Se puede usar `==`, `!=`, `<`, `>`, `<=`, `>=` (igual, distinto, menor, mayor, menor o igual, mayor o igual) o filtrar usando más de una condición¹. Usando esto:

¹Para más detalles capítulo 5.2 del Wickham

13. Seleccionen a los pasajeros mujeres y sobre esto encuentren la edad máxima. ¿Es el mismo resultado que calcular la edad máxima sobre todos los pasajeros?
14. Verificar: ¿qué edad máxima tienen los pasajeros considerados niños? ¿Es éste valor menor a la mínima edad de aquellos pasajeros adultos? **¡Ver cuál es la edad máxima de los niños!**

Gráficos de Barras

Después de una primera aproximación vamos a hacer algunos gráficos de barras (*bar plot*). Los gráficos de barras son útiles para mostrar **cantidades** y **proporciones**. Se generan de forma similar a los gráficos de dispersión de la guía anterior, llamando a `ggplot` primero indicando de dónde sacar los datos, luego indicando qué variable queremos agrupar para contar y por último llamando a `geom_bar()`. Por ejemplo, si queremos mostrar cuántos pasajeros hombres y mujeres había en el Titanic:

```
ggplot(data = titanic_df) +  
  geom_bar(aes(x=sex))
```

Cada una de las categorías de `sex` (*male* / *female*) va a ser una barra y la altura va a ser la **cantidad** de unidades que pertenecen a esa categoría. Pueden ver el resultado en la primera imagen de la Figura 1.

15. Elaborar gráficos de barras que permitan visualizar la cantidad de pasajeros según:
 - a. La cantidad de hermanos en el barco (variable `siblings.of.the.passenger`)
 - b. Si son hombres / mujeres / niños (variable `who`)

Hasta ahora estuvimos viendo cantidades de una sola variable (`sex`, `who`, etc.), pero también se puede aprovechar el diagrama de barras para mostrar los detalles de otra variable más. Por ejemplo, además de mostrar cuántos pasajeros de cada sexo había podemos ver cuántos había **en cada clase**.

Esto se puede hacer añadiendo una estética de relleno (*fill*) a las barras de sexo indicando qué atributo usar para **desagregar** cada barra. Por ejemplo, para ver el detalle de cada **clase** en el gráfico anterior agregamos `fill=class` al `aes`. Pueden ver la diferencia en la Figura 1:

```
ggplot(data = titanic_df) +  
  geom_bar(aes(x=sex, fill=class))
```

TIP: A veces trabajar con los nombres de columnas que tiene el *dataset* puede ser molesto, pueden cambiarlo usando `rename(data frame, 'nombre nuevo' = 'nombre viejo')`. Esto devuelve un nuevo *data frame* con el nuevo nombre de la columna.

Por ejemplo, cambiemos el nombre de la columna ‘Port.of.Embarkation’ a ‘Puerto’ y guardemos el nuevo *data frame* en `new_titanic_df`:

```
new_titanic_df <- rename(titanic_df, 'Puerto' = 'Port.of.Embarkation')
```

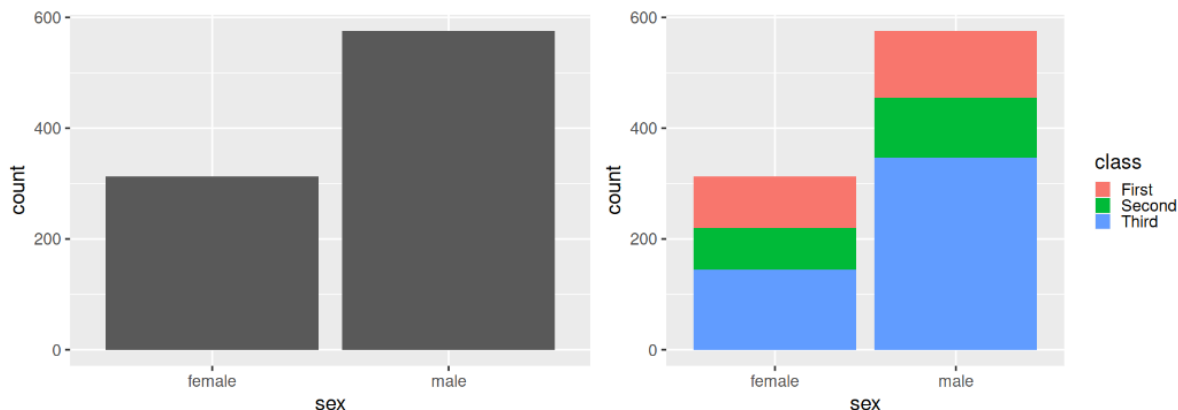


Figura 1: Cantidad de pasajeros hombres/mujeres y la misma información desagregada por clase.

16. Realizar **dos** gráficos de barras que indiquen la cantidad de pasajeros hombres, mujeres y niños (de forma desagregada) que viajaban en cada clase. Es decir, hagan un gráfico en función de las clases desagregando por hombre/mujer/niño (gráfico 1) y otro gráfico en función de hombre/mujer/niño, desagregando por clase (gráfico 2).
17. Mostrar en dos gráficos de barras la cantidad de personas que viajaban solas y las que viajaban acompañadas, desagregando por las variables “*who*” (gráfico 1) y “*class*” (gráfico 2).

Mirando el último de los gráficos del punto anterior (el primero de la Figura 2), ¿podemos decir a simple vista si entre los viajeros acompañados había más personas pertenecientes a segunda clase que entre los viajeros solitarios?

Si queremos hacer este tipo de comparaciones es útil desapilar las barras, lo que se logra agregando `position = "dodge"` al geom:

```
ggplot(data = new_titanic_df) +
  geom_bar(mapping = aes(x=alone, fill = class), position = "dodge")
```

En la Figura 2 se puede ver la comparación, y ahora sí podemos responder la pregunta. Hay **más** pasajeros de segunda clase viajando solos.



Figura 2: Desapilando para poder comparar individualmente

A veces nos resulta más útil en vez de hablar de cantidades hablar de porcentajes. Por ejemplo veamos el primer gráfico de Figura 3, es la cantidad de personas que viajan solas o acompañadas desagregado por **who**, que hacemos con la línea:

```
ggplot(data=new_titanic_df) +
  geom_bar(aes(x=alone, fill=who))
```

Se puede ver que **la cantidad** de mujeres que viajan solas es casi la misma que las que no, pero a simple vista no podemos ver qué relación tiene esto con la cantidad de personas dentro de cada categoría. O sea, no podemos responder una pregunta del estilo “¿De las personas que viajan solas qué **proporción** son mujeres?”

Para poder ver esto hay que agregar **position = "fill"** al geom:

```
ggplot(data = new_titanic_df) +
  geom_bar(mapping = aes(x=alone, fill = who), position = 'fill')
```

Ahora mirando el segundo gráfico de la Figura 3 sí podemos decir que “de las personas que viajan solas hay un 25% que son mujeres”.

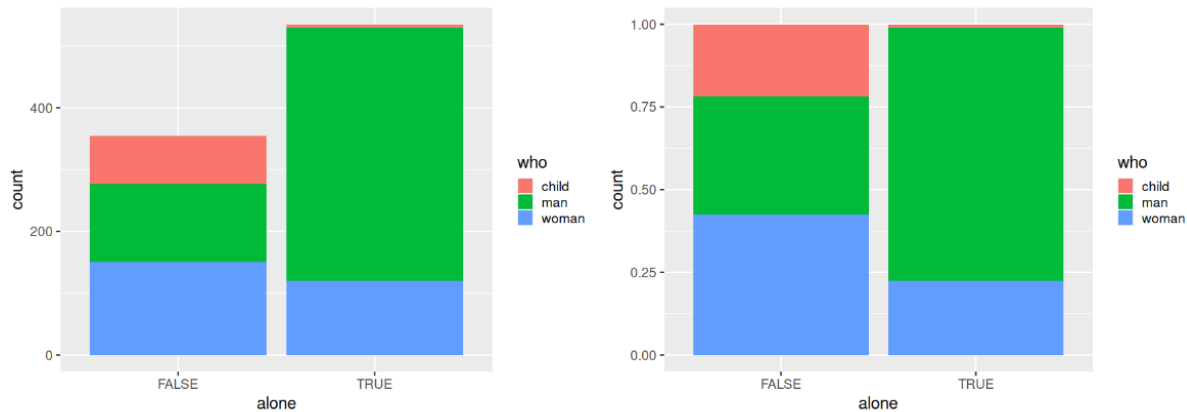


Figura 3: Cantidades y porcentajes

Hay un último detalle que quizás hayan notado, si bien cambió la escala en el segundo gráfico, sigue diciendo “count” (cantidad) esto no es del todo correcto. Para hacer gráficos más claros, recuerden que podemos cambiar el nombre a los ejes con `xlab(...)` e `ylab(...)`:

```
ggplot(data = new_titanic_df) +
  geom_bar(mapping = aes(x=alone, fill = class), position = "fill")+
  xlab("Viaja Solo") +
  ylab("Proporción")
```

El resultado se puede ver en la Figura 4.

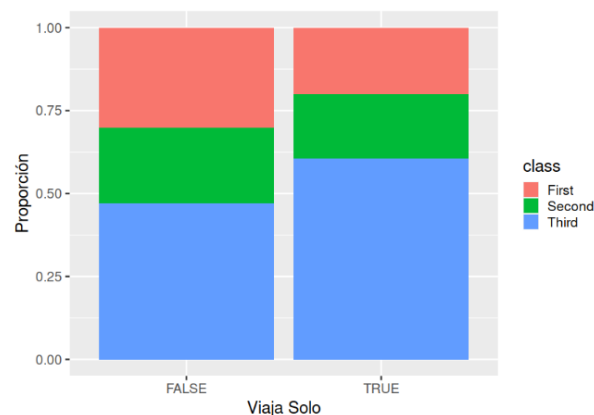


Figura 4: Proporciones de pasajeros de primera, segunda y tercera clase que viajan solos o acompañados

18. Se busca realizar un análisis sobre las características de los pasajeros en relación con el puerto de embarque. Realizar los gráficos de barras necesarios para responder a las siguientes preguntas. **Prestar especial atención al lugar que se asigna a cada variable para la elaboración del gráfico.**
- ¿Qué fracción de los pasajeros que embarcaron en el Puerto Southampton, Inglaterra eran mujeres?
 - ¿Desde qué puerto embarcó la mayor proporción de pasajeros de tercera clase?
 - ¿A qué clase pertenecía la mayor parte de los pasajeros que embarcaron en el puerto de Queenstown, Irlanda? (**Spoiler Alert:** esta pregunta y la anterior **NO** se pueden responder a partir del mismo gráfico).
 - De los pasajeros que embarcaron en el puerto de Southampton, Inglaterra, ¿la mayoría sobrevivió o falleció?
 - ¿Desde qué puerto embarcó la mayor cantidad de mujeres?
 - ¿Desde qué puerto embarcó la mayor proporción de mujeres?
 - Para los 3 puertos: encontrar la fracción de pasajeras mujeres que embarcaron desde allí.
19. Completar [este formulario](#).
20. Realizar el siguiente gráfico

```
ggplot(titanic_df) + geom_bar(aes(x=survived, fill=class), position = 'fill') +  
  labs(x = "¿Sobrevivió?", y = "Proporción") +  
  scale_x_discrete(labels=c('No', 'Sí')) +  
  scale_fill_manual(name = "Clase", labels = c("Primera", "Segunda", "Tercera"),  
                    values = c("tomato3", "darkseagreen3", "darkorchid"))
```

A partir de este gráfico, sin usar más información, decidir si las siguientes afirmaciones son verdaderas, falsas o si no es posible saberlo

- La mayoría de las personas de primera clase sobrevivieron.
- La mayor proporción de fallecidos corresponde a pasajeros de tercera clase.
- Sobrevivió más gente de primera clase que de segunda clase.

- d. De los pasajeros de segunda clase, la mayoría sobrevivió.
- e. La mayor fracción de sobrevivientes pertenece a primera clase.
- f. Hay más pasajeros de segunda clase que sobrevivieron que los que no sobrevivieron.
- g. Falleció una mayor parte de pasajeros de tercera clase que la que sobrevivió.

Si quieren repasar o explorar un poco más el tema siempre pueden recurrir el [Wickham](#) o a [este](#) link con un par de tips para agregar a las barras.

Agrupar

Ahora vamos a aprender a utilizar un comando que es de mucha utilidad para estudiar conjuntos de datos, ya que nos permite **resumir** o **colapsar** un grupo de valores en **un sólo** valor, calculado según la operación que especifiquemos. Se trata de `summarise()` o `summarize()`.

La idea es que `summarise()` toma un *data frame* y genera un nuevo *data frame* reducido en **una sola línea** que contiene únicamente las columnas que resultan de las operaciones de resumen que le indiquemos.

Dentro de `summarise()` podemos usar funciones de agregación como `mean()`, `sum()`, `min()`, `max()` y también funciones especiales de `dplyr`, como `n()`. La función `n()` devuelve la cantidad de filas del conjunto de datos que está siendo resumido:

Por ejemplo si ejecutan:

```
summarise(titanic_df, n())
```

Devuelve un nuevo *data frame* de una sola fila y una sola columna, cuyo valor es el total de filas en `titanic_df`.

! Nombre de columnas

En este ejemplo pueden ver que el nombre de la nueva columna con el resultado buscado es simplemente el nombre de la función (`'n()'`), pueden (o más bien, **deben**) asignarle un nombre más descriptivo, de esta forma:

```
summarise(titanic_df, cant_pasajeros = n())
```

Este comando suele aplicarse luego de haber **agrupado** las observaciones a partir de variables categóricas, utilizando la función `group_by()`.

Agrupar significa que el *data frame* se reorganiza internamente de manera que las filas que comparten el mismo valor en una o más columnas se consideren como parte del mismo “subconjunto” o grupo. Las operaciones de `summarise()` se aplican de forma independiente a cada uno de esos grupos.

Entonces, por ejemplo, si ejecutamos:

```
titanic_groupwho <- group_by(titanic_df, who)
```

`titanic_groupwho` es un nuevo *data frame* que internamente contiene la información agrupada según la variable “who”. Si lo abren en el entorno de trabajo, van a notar que visualmente se ve igual que `titanic_df`: el agrupamiento es algo interno que dplyr utiliza al momento de hacer cálculos, no una reordenación visible de las filas.

Ahora veamos qué pasa si aplicamos `summarise()` sobre este objeto agrupado:

```
summarise(titanic_groupwho, cant_pasajeros = n())
```

Verán en la consola que se imprime una tabla que muestra las variables en las cuales se agrupó al *dataset* y la cantidad de observaciones que pertenecen a ese grupo. Es decir, en este caso, la cantidad de pasajeros que pertenecían a la categoría “child”, “man” y “woman”. En general, todas las funciones que se apliquen a un *dataset* agrupado se aplicarán en **cada** grupo.

Veamos para qué nos puede ser útil. Si queremos simplemente calcular la edad promedio (con la función `mean`) de las personas que viajaban en el Titanic, podemos hacer:

```
summarise(titanic_df, edad.promedio = mean(age, na.rm=TRUE))
```

Esto nos devuelve un **único** valor, correspondiente al promedio de edad considerando todas las filas del *dataset*.

21. Realicen el mismo cálculo sobre el *dataset* agrupado creado arriba, `titanic_groupwho`. Si antes teníamos el promedio de todo el *dataset*, ¿qué estamos viendo acá?
22. Utilizando estos nuevos comandos (`group_by()`, `summarise()` y `n()`), calculen la cantidad de pasajeros que había:
 - a. En cada clase.

- b. Que hayan sobrevivido y que no.
- c. Que hayan subido en cada puerto.

Agrupar por más de una variable

La función `group_by()` también permite agrupar por varias columnas a la vez, separadas por comas. En ese caso, el *dataset* se divide internamente en **subgrupos definidos por todas las combinaciones posibles de esos valores**. Por ejemplo, ejecuten:

```
titanic_groupsex_class <- group_by(titanic_df, sex, class)
summarise(titanic_groupsex_class, cantidad = n())
```

Este código devuelve la cantidad de pasajeros para cada combinación de sexo y clase. Es decir, tendremos un conteo separado para hombre y mujeres de primera, segunda y tercera clase. Cuantas más variables usen para agrupar, más detallado será el desglose.

23. Calculen la edad promedio de los pasajeros según su sexo y si sobrevivieron o no. ¿Quiénes tenían, en promedio, más edad: los hombres que sobrevivieron o los que no? ¿y en el caso de las mujeres?.

Para estas últimas consignas también podrían haber utilizado el comando `count`. Investiguen cómo funciona, ejecútenlo y compárenlo con lo obtenido con `summarise()`. También pueden revisar la [sección del libro de Wickham](#).

24. Como habrán notado, la mayoría de las preguntas de esta Guía y todos los gráficos que tuvieron que elaborar involucraron variables categóricas. Aprendimos herramientas que, por ejemplo, nos permiten contar la cantidad de pasajeros por grupo según alguna variable categórica o determinar cómo se compone una variable categórica según alguna otra. ¿Podemos utilizar estas mismas herramientas para responder o hacer gráficos que aborden preguntas como las que siguen?
- a. ¿Qué clase era la más longeva?
 - b. ¿Cómo se relaciona el puerto de embarque con el precio del boleto?
 - c. Quienes sobrevivieron, en general, ¿eran menores o mayores que aquellos que fallecieron?
 - d. ¿Fue efectivamente la clase alta la que pagó los boletos más caros? ¿Esto ocurrió en todos los casos?
 - e. ¿De qué edad eran, en su mayoría, las mujeres? ¿Y los hombres?
 - f. ¿Qué edad tenían los pasajeros que viajaban solos (es decir, sin familiares)?

¿Se les ocurre cómo responder a estas preguntas? Realicen todas las pruebas necesarias con lo aprendido hasta ahora.

To be continued...

Resumen de lo visto en esta Guía:

- Exploración inicial: `view`, `spec`, `glimpse`.
- Acceso a columnas con `$`; valores extremos con `min`, `max` (y `na.rm=TRUE` para ignorar NA); valores únicos con `unique`.
- Filas extremas con `slice_min` / `slice_max` (argumento `n` para obtener más de una fila); reordenar columnas con `relocate`.
- Ordenar filas con `arrange` (ascendente y descendente con `desc`; por múltiples variables).
- Filtrar filas con `filter` (operadores `==`, `!=`, `<`, `>`, `<=`, `>=`; múltiples condiciones).
- Guardar resultados intermedios en variables usando `<-`.
- Renombrar columnas con `rename`.
- Gráficos de barras con `geom_bar`: desagregación con `fill`, posiciones `"stack"`, `"dodge"` y `"fill"` (proporciones); etiquetas con `xlab` / `ylab`.
- Agrupar con `group_by` y resumir con `summarise` (funciones `n()`, `mean()`, etc.); agrupar por múltiples variables.
- Conteo rápido con `count`.

Para trabajar en casa

Contestá el [Cuestionario de la Clase 2](#) antes de la clase virtual de esta semana.

Bibliografía obligatoria

Visualización de datos

Wickham: <https://es.r4ds.hadley.nz/03-visualize.html>

Cómo trabajar con bar plots en R

Ejemplos: <https://r-graph-gallery.com/barplot.html>

Etiquetas: https://www.data-to-viz.com/caveat/hard_label.html

Bibliografía para profundizar

Opción a los bar plots, los lollipop plot: <https://r-graph-gallery.com/lollipop-plot>

Si se puede, ordenar las barras: https://www.data-to-viz.com/caveat/order_data.html